



TRIVIA GAME

ANDERSON ARENAS
TITO BURBANO



INTRODUCTION

The Trivia Game Management System is designed to facilitate the administration and execution of a question-and-answer game. It has two components (Java (Spring Boot) and Python (FastAPI), both components use JSON for data storage and is the use of environment variables manage dynamic configurations

USER STORIES

- 1.As a player, I want to choose a question category (history, science, entertainment, etc.) so I can answer questions on a topic that interests me.
- 2.As a player, I want to earn rewards for correctly answering questions, so I can stay motivated and achieve achievements within the game.
- 3.As a player, I want the game to have a ranking system, so I can compare my performance to other players.
- 4.As an admin, I want to be able to add questions, so I can maintain the game alive.
- 5.As a player, I want to be able to log in within the game, so I can save my progress.
- 6.As a user, I want to search for all players, so I can see their information.
- 7.As a user, I want to search for a specific player, so I can see their information.
- 8.As an admin, I want to see all the questions that are in the game, so I can have that information at hand.
- 9.As an admin, I want to search for questions by a keyword, so I can check if it is already in the game
- 10.As a user I want to be able to create an account in the game, so I can play.
- 11.As a admin I want to be able to create an account in the game, so I can manage the game.
- 12.As a user I want to be able to log out so I can stop playing and save my progress.



FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements

User Management

- Users can register with a username, password, and initial score.
- Secure login/logout updates the user's online status.
- Admins can only be created by existing administrators.
- Users and admins can retrieve user data (e.g., leaderboard).

Question Management

- Users and admins can retrieve all or filtered questions.
- Admins can add new questions to the system.

Trivia Game Functionality

- Users can start a game session if logged in.
- The game presents random questions and updates scores.
- A leaderboard displays rankings after each session.

Non-Functional Requirements

Maintainability

- Uses a layered architecture (controllers, services, repositories).
- Dependency injection reduces tight coupling.

Portability

- JSON-based storage and environment variables support easy deployment.
- Java and Python components can run independently.

Performance

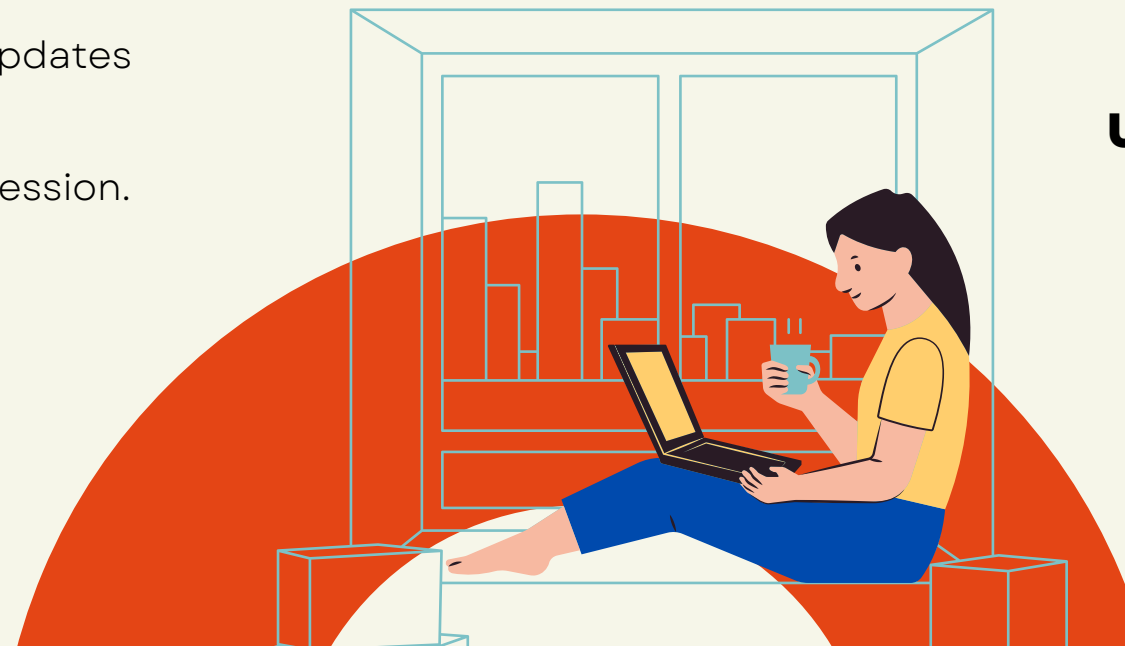
- JSON is efficient for small data volumes but may require a database later.

Security

- Passwords are stored in plain text (needs hashing like BCrypt).
- Token-based authentication (JWT) is recommended for future improvements.

Usability

- REST API is well-documented with Swagger/OpenAPI.
- User-friendly error handling improves the experience.



TECHNICAL CONSIDERATIONS

Architecture

- Java (Spring Boot) follows the MVC pattern with controllers for requests, services for business logic, and repositories for data.
- Python (FastAPI) uses APIRouter for endpoints, service classes for logic, and repositories for data handling.

Inter-Module Communication

- Java and Python components operate independently but share JSON data.
- Future integration may use RESTful APIs for direct communication.

Data Persistence

- Stores data in Users.json (user data) and Questions.json (question data).
- Uses environment variables for file paths, improving portability.

Session Management & User State

- Java marks users as "online" in JSON upon login, but this may not scale well.
- Python ensures users are online before starting a game session.

Environment & Configuration

- Uses environment variables to manage sensitive data like file paths.
- This approach enhances security and supports flexible deployments.



SOLID

Single Responsibility Principle

- Controllers/Routers handle HTTP requests.
- Services manage business logic (e.g., authentication, game sessions).
- Repositories handle data persistence with JSON.
- Factories (e.g., UserFactory) manage object creation.

Open/Closed Principle

- New features (e.g., a moderator role) can be added without modifying core logic.
- Services and repositories can be extended rather than changed.

Liskov Substitution Principle

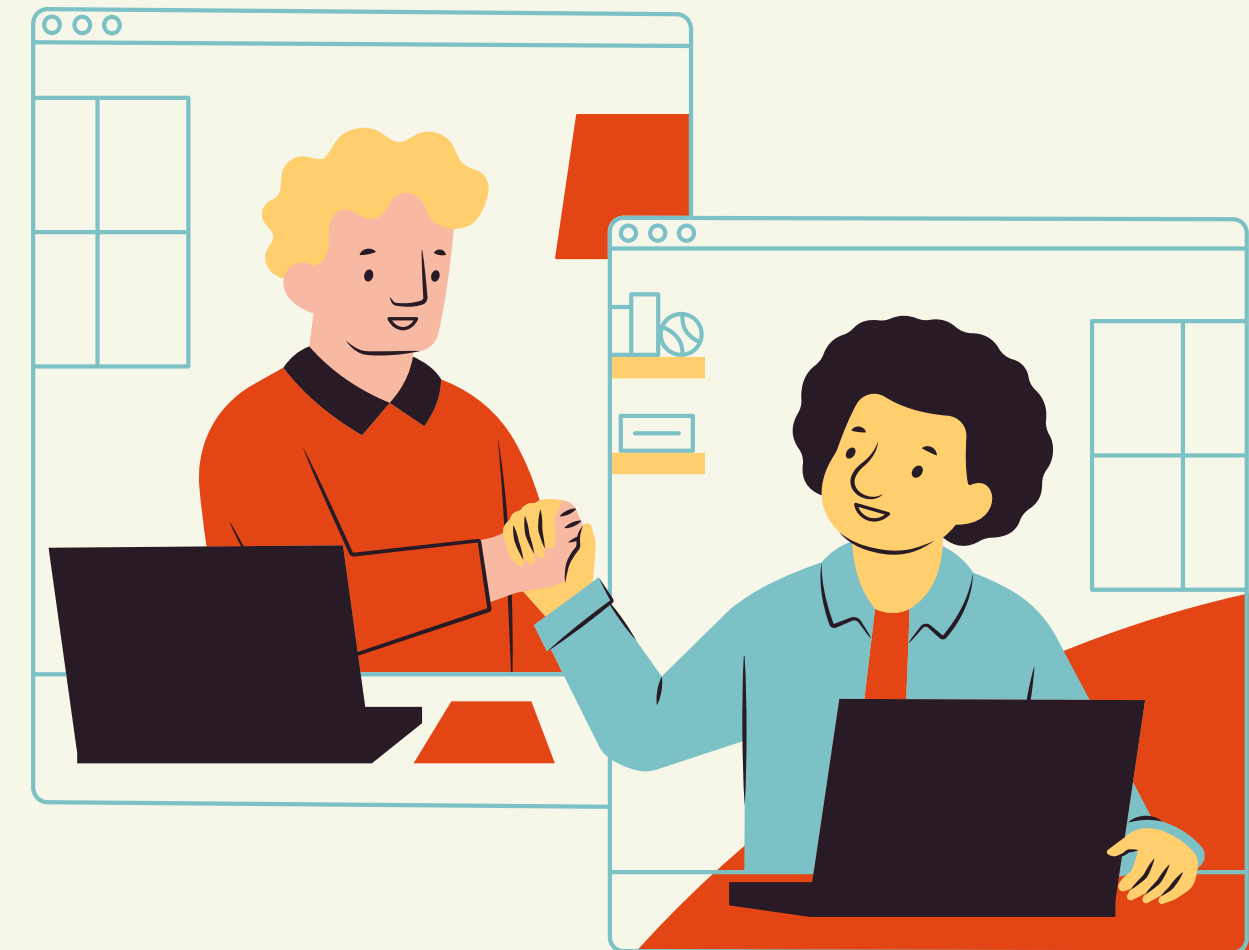
- Java: AdminUserDAO extends UserDAO without breaking functionality.

Interface Segregation Principle

- Modules expose only necessary methods, keeping interfaces clean.
- Controllers provide only relevant endpoints; services focus on specific business logic.

Dependency Inversion Principle

- Java: Uses Spring Boot's @Autowired for dependency injection.
- Python: Services receive repositories as parameters, keeping dependencies flexible.



DESIGN PATTERNS IN THE PROJECT

Factory Pattern

Purpose: Centralizes object creation to keep client code independent from concrete implementations.

Implementation:

- `UserFactory.createUser(...)` creates either `UserDAO` or `AdminUserDAO` based on user type.

Benefits:

- Centralized creation logic, making changes easier.
- Supports new user types without modifying controllers/services.
- Reduces dependencies on concrete implementations.

Facade Pattern

Purpose: Provides a unified interface to simplify interactions with complex subsystems.

Implementation:

- Java: `UserServices` acts as a bridge between `UserController` and `UserRepositories`.
- Python: `QuestionServices_Py` abstracts operations like filtering and validation.

Benefits:

- Simplifies client interactions with a clear interface.
- Decouples components, reducing the impact of changes.
- Improves maintainability and scalability.

Composite Pattern

Purpose: Treats individual objects and collections of objects uniformly, simplifying data management.

Implementation:

- Java: `UserRepositories` stores a collection of `UserDAO` instances.
- Python: `UserRepository_Py` and `QuestionRepository` manage collections of DAOs.

Benefits:

- Enables consistent operations on individual and grouped data.
- Centralizes data management for consistency and validation.
- Increases modularity and reusability.





PROGRAMMING BEST PRACTICES AND ANTI-PATTERNS

Best Practices

- **Layered Architecture:** Clear separation of concerns across controllers (routers), services, and repositories ensures modularity, maintainability, and testability.
- **Comprehensive Documentation:** JavaDoc for Java components and docstrings for Python modules enhance code clarity and onboarding for developers.
- **Dependency Injection:** Spring Boot's @Autowired and explicit repository instantiation in Python improve modularity and facilitate unit testing.
- **Use of Data Models:** Clearly defined domain entities (UserDAO, AdminUserDAO, QuestionDAO) ensure data integrity and validation (e.g., via Pydantic in Python).
- **Environment Configuration:** Use of environment variables for sensitive data increases security and portability.

Anti-Patterns and Risks

- **Plain-Text Password Storage:** Passwords are stored in plain text, posing a security risk. Implementing hashing (e.g., BCrypt) is crucial.
- **Basic Session Management:** JSON-based session tracking is simplistic and non-scalable. Moving to token-based authentication (JWT) or a dedicated session manager is recommended.
- **Reliance on JSON for Persistence:** JSON storage limits scalability and performance. Transitioning to a relational (e.g., PostgreSQL) or NoSQL database would improve efficiency.
- **Duplicated Logic Across Components:** Redundant models in Java and Python (UserDAO vs. UserDAO_Py) may cause inconsistencies. Consolidating shared logic can reduce maintenance issues.

CONCLUSIONS

1. Successful Multi-Language Integration

The project effectively integrates Java and Python components to manage a trivia game system. Despite the complexity of maintaining consistency across languages, the shared JSON data structure ensures smooth interaction and synchronization.

2. Effective Use of Design Patterns

- Factory Pattern: Centralizes object creation, enhancing flexibility.
- Facade Pattern: Simplifies interactions with complex subsystems.
- Composite Pattern: Enables uniform management of data collections.

These patterns improve modularity, maintainability, and scalability.

3. Adherence to SOLID Principles

- Strong single responsibility allocation across classes.
- Open/Closed Principle applied through extensible components.
- Dependency Inversion via service-repository separation.

This structured approach ensures robustness and adaptability.

4. Areas for Improvement

- Security Enhancements: Implement password hashing and adopt JWT for session management.
- Scalability Enhancements: Move from JSON storage to a database for better performance.
- Code Consolidation: Reduce duplication between Java and Python implementations.

5. Overall Impact

With a well-structured architecture, thoughtful design patterns, and adherence to best practices, the system is maintainable, scalable, and prepared for future enhancements or broader deployment.



THANKS

